Welcome to the
Web Application Development with Python L-4
Training Program

Md. Shakil - Assessor & Trainer - National Skill Development Authority (NSDA)

Lecture

10

Function in Python

Md. Shakil
Assessor & Trainer
National Skill Development Authority

What is Function?

A function is a reusable block of code that performs a specific task. Functions help in reducing code duplication and improving code organization.

- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result

Why used Functions?

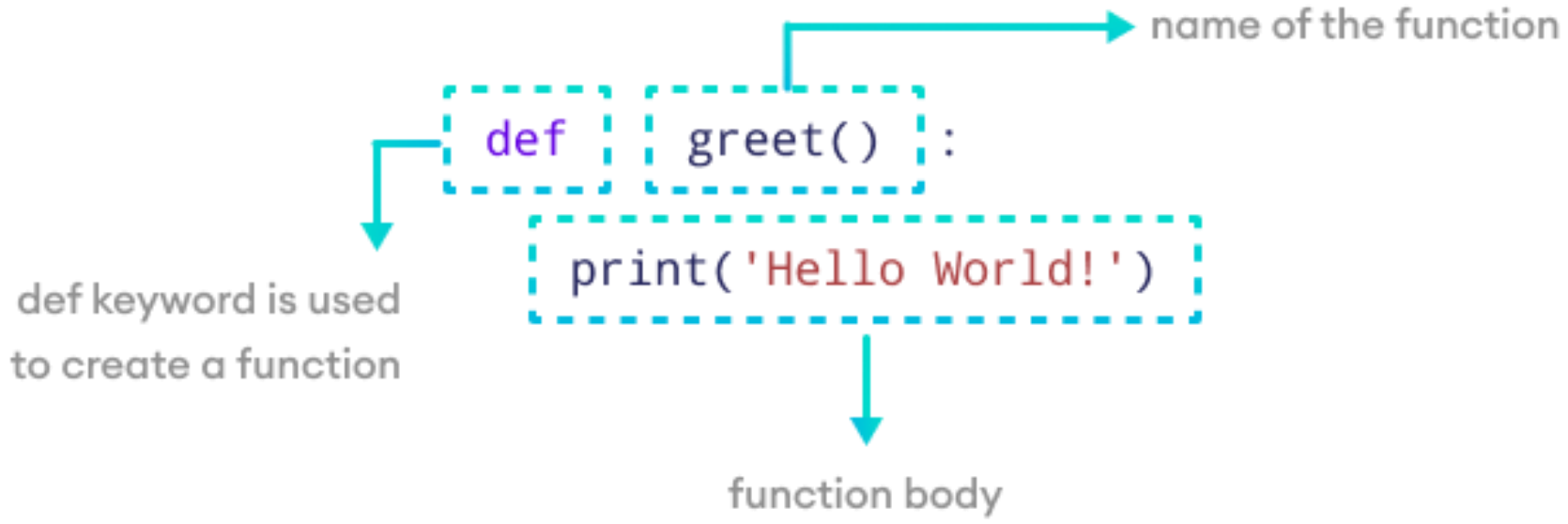
- Avoid code repetition
- Improve readability
- Make debugging easier
- Reuse logic multiple times
- Organize large programs

Types of Function

- **Built-in functions** (e.g., print(), len(), type())
- **User-defined functions** (functions you define yourself)

Create a Function

In Python a function is defined using the **def** keyword:



Calling a Function

To call a function, use the function name followed by parenthesis:

```
def greet():
  print('Hello World!')

# call the function
greet()

print('Outside function')
```

Python Function Parameters

A parameter is the variable listed inside the parentheses in the function definition.

function.py

```python
def greet(name):
    print("Hello", name)

# pass argument
greet("John")
```

Parameter

Function with Default Parameters

Default parameter values are used if no argument is provided.

function.py

```python
def greet(name="Guest"):
    print(f"Hello, {name}!")

greet()            # Output: Hello, Guest!
greet("Rohim")    # Output: Hello, Rohim!
```

Function with Multiple Parameters

```python
def add(a, b):
    print("Sum =", a + b)

add(10, 20)
```

Python Function Argument

- In computer programming, an argument is a value that is accepted by a function.
- Arguments are inputs given to the function.
- Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma.

****Arguments** are often shortened to ***args*** in Python documentations.

function.py

```python
def greet(name):
    print("Hello", name)

# pass argument
greet("John")
```

Argument

Number of Arguments

By default, a function must be called with the correct number of arguments. Meaning that if your function expects 2 arguments, you have to call the function with 2 arguments, not more, and not less.

function.py

```
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Mr.", "Rohim")
```

Python Keyword Arguments

- In keyword arguments, arguments are assigned based on the name of the arguments.
- You can also send arguments with the ***key = value* syntax**.
- This way the order of the arguments does not matter.

function.py

```python
def student_info(name, age):
    print(f"Name: {name}, Age: {age}")

student_info(age=20, name="Amina")
```

Function Argument with Default Values

function.py

```python
def add_numbers( a = 7,  b = 8):
    sum = a + b
    print('Sum:', sum)

# function call with two arguments
add_numbers(2, 3)  #Output: Sum: 5

#  function call with one argument
add_numbers(a = 2)  #Output: Sum: 10

# function call with no arguments
add_numbers()  #Output: Sum: 15
```

Arbitrary Arguments, *args

- If you do not know how many arguments that will be passed into your function, add a * before the parameter name in the function definition.
- This way the function will receive a **tuple** of arguments, and can access the items accordingly:

function.py

```python
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

function.py

```python
def find_sum(*numbers):
    result = 0
    for num in numbers:
        result = result + num
    print("Sum = ", result)

# function call with 3 arguments
find_sum(1, 2, 3)
# function call with 2 arguments
find_sum(4, 9)
```

Arbitrary Keyword Arguments, **kwargs

- If you do not know how many keyword arguments that will be passed into your function, add two asterisk: ** before the parameter name in the function definition.
- This way the function will receive a **dictionary** of arguments, and can access the items accordingly:

function.py

```
def my_function(**kid):
  print("His last name is " + kid["lname"])

my_function(fname = "Tobias", lname = "Refsnes")
```

The return Statement

We return a value from the function using the return statement.

function.py

```python
# function definition
def find_square(num):
    result = num * num
    return result

# function call
square = find_square(3)

print('Square:', square)
```

The pass Statement

The pass statement serves as a placeholder for future code, preventing errors from empty code blocks.

It's typically used where code is planned but has yet to be written.

function.py

```python
def myfunction():
  pass
```

Thanks All